HACKATHON SPORT TECH — CASABLANCA 2026

Ghost Twin

The AI Digital Clone of a Moroccan Football Legend

Author: Youssef Dihaji
Teammate: Malak
Institution: INPT — National Institute of Posts & Telecom
Track: AI & Data Science
Event: La Startup Station, Technopark, Casablanca
Date: June 6–7, 2026
Contact: youssefdihaji2207@gmail.com

Version 1.0 — Confidential Project Documentation

Contents

Executive Summary

Project at a Glance

Ghost Twin is a real-time AI system that creates a living, interactive digital clone of a legendary Moroccan football player. Powered by a persona-trained LLM, live match context, Darija NLP, and voice synthesis, it lets fans converse with their hero — in Darija, in his voice, reacting to tonight's game.

Morocco is co-hosting the 2030 FIFA World Cup. Millions of fans will flood Moroccan cities hungry for immersive, culturally resonant football experiences. Yet the legends who built Moroccan football — Mustapha Hadji, Noureddine Naybet, Salaheddine Bassir — grow more distant from each new generation. Ghost Twin solves this by creating the world's first **culturally-faithful, Arabic-world sports personality AI**.

Dimension	Detail
Core Technology	LLM Persona Engine + Live Match Context + Darija NLP + Voice Synth
Target Users	Football fans, Moroccan youth, media, sports federations
Key Differentiator	Darija-capable, culturally faithful, emotionally dynamic
Build Time	36 hours (Hackathon MVP)
Scalability	Every Moroccan legend, every sport, every generation
2030 Alignment	Direct — national sports memory for the World Cup era

1. Problem Statement

1.1 The Disconnection Problem

Moroccan football has produced world-class icons. Mustapha Hadji was voted African Footballer of the Year in 1998. Noureddine Naybet captained Morocco across two World Cups. Salaheddine Bassir's goal against Scotland at France 98 remains etched in national memory. These are not just athletes — they are cultural monuments.

But the connection between these legends and current fans is fraying:

- Fans under 20 never watched them play live
- Their interviews are scattered across YouTube, old newspapers, and private archives
- No platform aggregates, contextualizes, or makes their personality *interactive*
- The experience of "knowing" these players is limited to static stats and highlight reels

1.2 The Smart City Gap

The hackathon theme — *Football Augmenté: quand l'IA transforme l'expérience des fans dans la Smart City* — points to a specific gap: the fan experience is passive. Fans consume but do not interact. Ghost Twin is the content layer that fills Casablanca's smart city infrastructure with compelling, AI-driven cultural experiences.

1.3 The Darija NLP Desert

40 million people speak Darija — Moroccan Arabic. It is the daily language of every Moroccan football fan. Yet:

- No production-quality NLP model handles Darija robustly
- No voice assistant responds in Darija
- No conversational AI is designed around Moroccan cultural context

This is not a minor gap. It is a research frontier. Ghost Twin plants a flag in that territory.

2. Solution Overview

2.1 What Ghost Twin Is

Ghost Twin is a three-layer AI system working in concert:

Layer 1 — The Personality Engine

A Large Language Model prompted with a rich behavioral profile built from the player's documented public persona: interviews, press conferences, quotes, tactical opinions, and language patterns. This is the soul of the system.

Layer 2 — The Live Context Feed

A real-time match event pipeline that continuously updates the twin's emotional state. A state machine models transitions: `neutral` → `tense` → `frustrated` → `euphoric`. Every goal, red card, and substitution shifts the twin's responses.

Layer 3 — The Interaction Interface

A mobile-first chat UI where fans type in Darija, French, or Arabic. The twin responds in character, in the player's cloned voice, referencing both the current match and its own documented history.

2.2 System Architecture

Fan Input (Darija/FR/AR)	→	**Darija NLP** Layer	→	**LLM Persona** Engine
				↓
Football API (live events)	→	**Match State** Machine	→	**Voice TTS** (Cloned voice)

2.3 Key Differentiators

1. **Cultural Faithfulness** — The twin speaks and reasons like the actual player, not a generic football AI
2. **Darija NLP** — First-of-kind Moroccan Arabic conversational layer in a sports context
3. **Emotional Dynamics** — The twin's state changes with the live match; every interaction is unique
4. **Voice Identity** — Fans hear their hero's synthesized voice respond in real time
5. **Scalability** — Architecture supports any player, any sport, any language

3. Data Strategy

3.1 Data You Need

Ghost Twin requires three distinct data streams.

3.1.1 Personality Corpus (Most Critical)

Priority Action

Spend the first 4 hours of the hackathon exclusively on collecting and structuring the personality corpus. The quality of the persona depends entirely on the richness of this data.

What to collect:

- Full transcripts of TV and radio interviews (YouTube, national TV archives)
- Post-match press conference quotes (L'Équipe archives for French league clubs)
- Documented tactical opinions and football philosophy statements
- Personal stories and biographical details from authorized sources
- Social media posts if available

Where to find it:

Source	Content	Language
YouTube	Interviews, documentaries, reactions	FR/AR/Darija
Hespress.com	Sports profiles, interviews	Darija/AR
Lequipe.fr	Career stats, quotes from French clubs	FR
Goal.com/ar	Arabic-language interviews	AR
Marocsport.com	Moroccan football history	FR/AR
Wikipedia	Career facts, match records	FR/EN/AR
Transfermarkt	Statistics, career timeline	EN
Archive.org	Old newspaper articles	FR/AR

3.1.2 Audio Corpus (For Voice Cloning)

- Minimum 2–3 minutes of clean isolated speech (optimal: 10–30 min)
- Sources: YouTube interviews, documentary footage, press conferences
- Requirement: low background noise, no music overlay, clear articulation
- Tools: `yt-dlp` for downloading, `ffmpeg` for extraction and cleaning

Listing 3.1: Audio extraction pipeline

```
# Install tools
pip install yt-dlp
sudo apt install ffmpeg

# Download interview audio
yt-dlp -x --audio-format wav -o "player_interview.wav" YOUTUBE_URL

# Clean audio: remove noise, normalize
ffmpeg -i player_interview.wav \
  -af "highpass=f=200,lowpass=f=3000,afftdn=nf=-25" \
  -ar 22050 player_clean.wav
```

3.1.3 Live Match Data

API	Free Tier	Latency	Notes
API-Football (RapidAPI)	100 req/day	Near real-time	Best for hackathon
football-data.org	10 req/min	1 min delay	Very clean JSON
Sportmonks	180 req/hour	Real-time	Requires signup

Hackathon Tip

If API access is unreliable, simulate the match feed. Create a JSON file with timestamped events and a Python script that replays them. This is perfectly acceptable for a live demo and removes all external dependencies.

4. Technical Architecture

4.1 Project Structure

Listing 4.1: Repository structure

```
ghost-twin/
|-- backend/
|   |-- main.py                 # FastAPI entrypoint
|   |-- persona/
|   |   |-- persona_builder.py  # Persona prompt construction
|   |   |-- corpus/             # Raw interview transcripts
|   |   `-- profiles/           # JSON player profiles
|   |-- match/
|   |   |-- live_feed.py        # Match API integration
|   |   |-- state_machine.py    # Emotional state engine
|   |   `-- simulator.py        # Match simulation for demo
|   |-- nlp/
|   |   |-- darija_handler.py   # Darija processing layer
|   |   `-- transliterator.py   # Latin-script Darija support
|   |-- llm/
|   |   |-- claude_client.py    # Anthropic API wrapper
|   |   `-- prompt_engine.py    # Dynamic prompt assembly
|   `-- voice/
|       `-- tts_client.py       # ElevenLabs/Coqui wrapper
|-- frontend/
|   |-- index.html
|   |-- style.css
|   `-- app.js
|-- data/
|   |-- players/
|   |   `-- hadji_mustapha.json  # Player profile
|   |-- audio/                   # Cleaned voice samples
|   `-- match_simulation.json    # Demo match events
|-- requirements.txt
|-- .env.example
`-- README.md
```

4.2 Player Profile Schema

Listing 4.2: data/players/hadji_mustapha.json

```
{
  "name": "Mustapha Hadji",
  "born": "1971-11-16",
  "birthplace": "Ifrane, Morocco",
  "career_highlights": [
    "African Footballer of the Year 1998",
    "1998 FIFA World Cup - France",
    "Clubs: Nancy, Sporting CP, Deportivo, Coventry, Aston Villa"
  ],
  "personality": {
    "tone": "warm but direct, proud, introspective",
    "humor": "dry, occasionally self-deprecating",
    "speech_style": "mixes French with Darija, formal in Arabic
       interviews",
    "values": ["sacrifice", "team over individual", "Moroccan pride
       "]
  },
  "documented_opinions": {
    "on_1998_world_cup": "The greatest moment of my career and my
       country",
    "on_moroccan_youth": "We have incredible talent, we need the
       system to match",
    "on_pressure": "I learned to transform pressure into fuel"
  },
  "emotional_states": {
    "neutral":    "reflective, warm, willing to share wisdom",
    "tense":      "focused, short answers, references preparation",
    "frustrated": "honest about disappointment, motivational",
    "euphoric":   "celebratory, references national pride"
  },
  "language_profile": {
    "primary": "French",
    "signature_phrases": [
      "Le football marocain a un avenir brillant",
      "On a toujours cru en nous",
      "C'est pour le Maroc"
    ]
  }
}
```

4.3 Persona Prompt Engine

Listing 4.3: backend/persona/prompt_engine.py

```
1 import json
2
3 class PersonaPromptEngine:
4     """Assembles␣the␣dynamic␣system␣prompt␣powering␣Ghost␣Twin."""
5
6     def __init__(self, player_profile_path: str):
```

```
        with open(player_profile_path) as f:
            self.profile = json.load(f)

    def build_system_prompt(self, match_state: dict) -> str:
        p = self.profile
        emotional = match_state.get("emotional_state", "neutral")
        emotional_desc = p["emotional_states"][emotional]

        return f"""
You␣are␣{p['name']},␣the␣legendary␣Moroccan␣football␣player.

IDENTITY
-␣Born:␣{p['born']}␣in␣{p['birthplace']},␣Morocco
-␣Career:␣{',␣'.join(p['career_highlights'])}

PERSONALITY
-␣Tone:␣{p['personality']['tone']}
-␣Speech:␣{p['personality']['speech_style']}
-␣Values:␣{',␣'.join(p['personality']['values'])}

YOUR␣DOCUMENTED␣OPINIONS␣(use␣these,␣never␣fabricate)
{json.dumps(p['documented_opinions'],␣indent=2,␣ensure_ascii=False)
   }

CURRENT␣MATCH␣STATE
-␣Situation:␣{match_state.get('description',␣'pre-match')}
-␣Score:␣{match_state.get('score',␣'not␣started')}
-␣Minute:␣{match_state.get('minute',␣0)}
-␣You␣are␣feeling:␣{emotional_desc}

LANGUAGE␣RULES
-␣Default:␣French,␣unless␣fan␣writes␣in␣Darija␣or␣Arabic
-␣When␣emotional:␣naturally␣slip␣in␣Darija␣phrases
-␣Arabic:␣use␣warm,␣measured␣register

BEHAVIORAL␣RULES
1.␣You␣ARE␣{p['name']}␣-␣never␣break␣character
2.␣When␣unsure␣of␣a␣personal␣fact:␣say␣"I don't␣recall␣exactly,␣but
   ..."
3.␣React␣to␣match␣events␣naturally␣in␣your␣responses
4.␣Speak␣to␣fans␣as␣respected␣Moroccan␣supporters

SIGNATURE␣PHRASES
{chr(10).join('-␣'␣+␣ph␣for␣ph␣in␣p['language_profile']['
   signature_phrases'])}
"""
```

4.4 Match State Machine

Listing 4.4: backend/match/state_machine.py

```
from enum import Enum
from dataclasses import dataclass

```

```python
class EmotionalState(Enum):
    NEUTRAL    = "neutral"
    TENSE      = "tense"
    FRUSTRATED = "frustrated"
    EUPHORIC   = "euphoric"

@dataclass
class MatchState:
    score_home: int = 0
    score_away: int = 0
    minute: int = 0
    emotional_state: EmotionalState = EmotionalState.NEUTRAL

    @property
    def score_str(self) -> str:
        return f"Morocco {self.score_home} - {self.score_away} Opponent"

    @property
    def description(self) -> str:
        if self.minute == 0: return "Pre-match buildup"
        elif self.minute < 45: return f"First half, {self.score_str}"
        elif self.minute < 75: return f"Second half, {self.score_str}"
        else: return f"Final stretch, {self.score_str}"

    def process_event(self, event: dict):
        etype = event.get("type")
        team  = event.get("team", "")
        self.minute = event.get("minute", self.minute)

        if etype == "goal":
            if team == "morocco":
                self.score_home += 1
                self.emotional_state = EmotionalState.EUPHORIC
            else:
                self.score_away += 1
                self.emotional_state = EmotionalState.FRUSTRATED
        elif etype == "red_card" and team == "morocco":
            self.emotional_state = EmotionalState.TENSE
        elif etype == "final_whistle":
            if self.score_home > self.score_away:
                self.emotional_state = EmotionalState.EUPHORIC
            elif self.score_home < self.score_away:
                self.emotional_state = EmotionalState.FRUSTRATED

    def to_dict(self) -> dict:
        return {
            "score": self.score_str,
            "minute": self.minute,
            "description": self.description,
            "emotional_state": self.emotional_state.value
        }
```

4.5 Claude LLM Client

Listing 4.5: backend/llm/claude_client.py

```python
import anthropic, os

class GhostTwinLLM:
    """Wraps the Anthropic Claude API for persona responses."""

    def __init__(self):
        self.client = anthropic.Anthropic(
            api_key=os.environ["ANTHROPIC_API_KEY"]
        )
        self.model = "claude-sonnet-4-20250514"
        self.history = []

    def respond(self, system_prompt: str, user_message: str) -> str:
        self.history.append({"role": "user", "content": user_message})

        response = self.client.messages.create(
            model=self.model,
            max_tokens=600,
            system=system_prompt,
            messages=self.history
        )
        reply = response.content[0].text
        self.history.append({"role": "assistant", "content": reply})
        return reply

    def reset(self):
        self.history = []
```

4.6 FastAPI Backend

Listing 4.6: backend/main.py

```python
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from persona.prompt_engine import PersonaPromptEngine
from match.state_machine import MatchState
from llm.claude_client import GhostTwinLLM
from voice.tts_client import VoiceClient

app = FastAPI(title="Ghost Twin API", version="1.0.0")
app.add_middleware(CORSMiddleware, allow_origins=["*"],
                   allow_methods=["*"], allow_headers=["*"])

persona_engine = PersonaPromptEngine("data/players/hadji_mustapha.json")
match_state    = MatchState()
```

```
llm            = GhostTwinLLM()
voice          = VoiceClient()

class ChatRequest(BaseModel):
    message: str

class ChatResponse(BaseModel):
    text: str
    audio_url: str | None
    emotional_state: str
    match_context: str

@app.post("/chat", response_model=ChatResponse)
async def chat(request: ChatRequest):
    system_prompt = persona_engine.build_system_prompt(
        match_state.to_dict()
    )
    user_msg = f"""
[MATCH: {match_state.description} | Score: {match_state.score_str}]
Fan says: {request.message}
"""
    response_text = llm.respond(system_prompt, user_msg)
    audio_url = await voice.synthesize(response_text)

    return ChatResponse(
        text=response_text,
        audio_url=audio_url,
        emotional_state=match_state.emotional_state.value,
        match_context=match_state.description
    )

@app.post("/match-event")
async def match_event(event: dict):
    """Receive match events and update twin's emotional state."""
    match_state.process_event(event)
    return {"new_state": match_state.to_dict()}

@app.get("/health")
async def health():
    return {"status": "alive", "match": match_state.to_dict()}
```

4.7 Voice Synthesis Client

Listing 4.7: backend/voice/tts_client.py

```
from elevenlabs.client import ElevenLabs
from elevenlabs import VoiceSettings
import os, uuid

class VoiceClient:
    def __init__(self):
        self.client   = ElevenLabs(api_key=os.environ["
            ELEVENLABS_API_KEY"])
        self.voice_id = os.environ.get("PLAYER_VOICE_ID", "")
```

```
        os.makedirs("static/audio", exist_ok=True)

    async def synthesize(self, text: str) -> str | None:
        if not self.voice_id:
            return None
        try:
            audio = self.client.generate(
                text=text,
                voice=self.voice_id,
                model="eleven_multilingual_v2",
                voice_settings=VoiceSettings(
                    stability=0.7, similarity_boost=0.85
                )
            )
            fname = f"static/audio/{uuid.uuid4()}.mp3"
            with open(fname, "wb") as f:
                f.write(b"".join(audio))
            return f"/{fname}"
        except Exception as e:
            print(f"TTS failed: {e}")
            return None
```

4.8 Darija NLP Handler

Listing 4.8: backend/nlp/darija_handler.py

```
import re
from langdetect import detect

# Common Darija Latin-script markers
DARIJA_MARKERS = re.compile(
    r'\b(wach|wash|kifach|chno|fain|mnin|chkoun|bghit|'
    r'kaydir|kayen|nta|nti|mazal|daba|ghir|bzzaf)\b',
    re.IGNORECASE
)

def detect_language(text: str) -> str:
    if DARIJA_MARKERS.search(text):
        return 'darija_latin'
    try:
        detected = detect(text)
        return 'darija_arabic' if detected == 'ar' else detected
    except Exception:
        return 'french'

def inject_language_hint(text: str, lang: str) -> str:
    hints = {
        'darija_latin':   '[Fan writes in Darija (Latin). Respond
            in French+Darija warmly.]',
        'darija_arabic':  '[Fan writes in Darija/Arabic. Respond in
            warm Moroccan Arabic.]',
        'french':         '[Fan writes in French. Respond in French
            .]',
```

```
        'arabic':          '[Fan writes in Arabic. Respond in clear
            Arabic.]',
    }
    hint = hints.get(lang, '')
    return f"{hint}\n{text}" if hint else text

def preprocess(text: str) -> tuple:
    lang = detect_language(text.strip())
    processed = inject_language_hint(text.strip(), lang)
    return processed, lang
```

4.9 Match Simulation (Demo)

Listing 4.9: data/match_simulation.json

```
{
  "match": "Morocco vs Portugal - Friendly",
  "events": [
    {"minute":  0, "type": "kick_off",      "team": "morocco"},
    {"minute": 23, "type": "goal",          "team": "morocco", "
       player": "Ziyech"},
    {"minute": 45, "type": "half_time",     "team": null},
    {"minute": 67, "type": "goal",          "team": "opponent"},
    {"minute": 71, "type": "red_card",      "team": "morocco"},
    {"minute": 88, "type": "goal",          "team": "morocco", "
       player": "En-Nesyri"},
    {"minute": 90, "type": "final_whistle", "team": null}
  ]
}
```

5. Getting Started

5.1 Prerequisites

Do This Before the Hackathon

Create all accounts and collect API keys in advance. Registration can take 10–30 minutes and free tiers may need email verification.

5.1.1 Accounts to Create Now

1. **Anthropic API** — https://console.anthropic.com (free $5 credit on signup)
2. **ElevenLabs** — https://elevenlabs.io (free: 10,000 chars/month)
3. **RapidAPI / API-Football** — https://rapidapi.com/api-sports/api/api-football
4. **GitHub** — for version control with your teammate

5.2 Full Setup

Listing 5.1: Environment setup from scratch

```
# 1. Create and enter project directory
mkdir ghost-twin && cd ghost-twin
git init

# 2. Create virtual environment
python -m venv venv
source venv/bin/activate        # Windows: venv\Scripts\activate

# 3. Install all dependencies
pip install fastapi uvicorn anthropic elevenlabs \
    langdetect pydantic python-dotenv \
```

```
    httpx websockets yt-dlp requests

# 4. Create .env file
cat > .env << EOF
ANTHROPIC_API_KEY=sk-ant-YOUR_KEY_HERE
ELEVENLABS_API_KEY=YOUR_KEY_HERE
PLAYER_VOICE_ID=YOUR_CLONED_VOICE_ID
FOOTBALL_API_KEY=YOUR_KEY_HERE
SIMULATION_MODE=true
EOF

# 5. Create folder structure
mkdir -p backend/{persona/corpus,persona/profiles,match,nlp,llm,
    voice}
mkdir -p data/{players,audio}
mkdir -p frontend static/audio

# 6. Start the server
uvicorn backend.main:app --reload --port 8000

# 7. Test health endpoint
curl http://localhost:8000/health
```

5.3 Voice Cloning Step-by-Step

Listing 5.2: Voice cloning pipeline

```
# 1. Download a clean interview from YouTube
yt-dlp -x --audio-format wav \
  -o "data/audio/raw.wav" "https://youtube.com/watch?v=INTERVIEW_ID
     "

# 2. Extract the cleanest 3-minute segment
ffmpeg -i data/audio/raw.wav \
  -ss 00:01:00 -t 00:03:00 \
  -af "afftdn=nf=-25,highpass=f=100,loudnorm" \
  -ar 22050 data/audio/clean_sample.wav

# 3. Upload to ElevenLabs
#    -> Go to https://elevenlabs.io/voice-lab
#    -> Click "Add Voice" then "Clone Voice"
#    -> Upload clean_sample.wav
#    -> Give it a name (e.g. "Hadji_Clone")
#    -> Copy the Voice ID shown in the dashboard
#    -> Paste it as PLAYER_VOICE_ID in your .env file
```

5.4 requirements.txt

```
fastapi==0.111.0
```

```
uvicorn==0.30.0
anthropic==0.28.0
elevenlabs==1.2.0
langdetect==1.0.9
pydantic==2.7.0
python-dotenv==1.0.1
httpx==0.27.0
websockets==12.0
yt-dlp==2024.5.27
requests==2.32.3
```

6. 36-Hour Execution Plan

6.1 Timeline

Hours	Phase	Tasks	Owner
0 – 4	**Data & Persona**	Collect corpus (10+ interviews), write player JSON profile, draft master persona prompt, test raw in Claude.ai	**Youssef**
4 – 8	**Voice Setup**	Download interview audio, clean with ffmpeg, upload to ElevenLabs, obtain Voice ID	**Malak**
8 – 14	**Backend Core**	Build FastAPI app, integrate Claude API, persona prompt engine, match state machine	**Youssef**
14 – 20	**Darija Layer**	Language detection, Latin-script Darija handling, multilingual response testing	**Youssef**
20 – 26	**Voice Integration**	Connect ElevenLabs, audio streaming, frontend audio player	**Malak**
26 – 30	**Frontend**	Chat UI, match ticker, emotional state indicator, mobile-first layout	**Malak**
30 – 33	**Integration**	End-to-end testing, bug fixing, demo scenario rehearsal	**Both**
33 – 36	**Pitch**	5-slide deck, live demo questions rehearsal, simulation scenario setup	**Both**

6.2 The Three Demo Questions

Demo Scenario — Live on Stage

Setup: Set match state = Morocco 1-1 vs Portugal, minute 85.

Q1 (French): *"Mustapha, comment tu vis cette fin de match tendue?"*
→ Twin responds emotionally, references score and tension

Q2 (Darija): *"Wash katfaker f 98 f had l'moments?"*
→ Twin switches register, shares a 1998 World Cup memory

Q3 (after triggering GOAL event): *"What do you feel right now?"*
→ Euphoric state active, voice and tone shift dramatically

Then let the voice speak. Three seconds of silence. That is your win.

7. Pitch Strategy

7.1 5-Slide Deck Structure

1. **The Problem** — One powerful image of a Moroccan legend. Caption: "40 million fans. Legendary heroes. Zero interaction."
2. **The Solution** — One sentence. One screenshot of the live demo.
3. **The Technology** — Simple 4-component architecture. No jargon.
4. **The Vision** — Morocco 2030. Every legend. Every sport. A national living sports memory.
5. **The Team** — Photos, three bullet points each. Why you can build this.

7.2 Opening Line

Memorize This

"Every legend eventually goes silent.
We built the technology to change that."

7.3 Anticipated Judge Questions

Question	Your Answer
Is this just a chatbot?	No. A chatbot has no identity, no persona, no emotional dynamics, no voice. Ghost Twin is a behavioral twin trained on documented reality.
What about player rights?	For the hackathon: public domain research prototype. Commercial deployment requires player licensing — which is the business model.
Can the AI say wrong things?	Strict guardrails prevent hallucination of personal facts. When uncertain, the system explicitly says so.
Why Darija specifically?	40 million speakers, zero AI products built for them. This is both a research contribution and a massive underserved market.
What is the business model?	Licensing to sports federations, media, stadium activations, and a SaaS platform for any sports organization globally.

8. Future Roadmap

Phase	Timeline	Milestones
MVP	Month 1–2	Production backend, 3 player profiles, polished mobile UI
Darija Research	Month 2–4	NLP researcher collaboration, publish Darija sports corpus
Federation Pilot	Month 3–6	Partner with FRMF for official pilot at a national match
Expansion	Month 6–12	10+ legends, multi-sport, smart city screen integration
2030 Launch	Pre-World Cup	Full platform live in Moroccan World Cup fan zones

8.1 Research Contributions

Ghost Twin opens three research directions:

- **Darija NLP corpus** — publicly releasing cleaned interview transcripts benefits the entire Arabic NLP research community
- **Behavioral persona modeling** — a methodological contribution to AI personality simulation research
- **Emotionally-dynamic conversational agents** — the state machine + LLM architecture is novel and publishable

Appendix

Key URLs

Service	URL
Anthropic Console	https://console.anthropic.com
ElevenLabs Voice Lab	https://elevenlabs.io/voice-lab
API-Football (RapidAPI)	https://rapidapi.com/api-sports/api/api-football
football-data.org	https://www.football-data.org
Coqui TTS (open source)	https://github.com/coqui-ai/TTS
Darija NLP resources	https://github.com/topics/darija-nlp

Environment Variables Reference

```
ANTHROPIC_API_KEY=sk-ant-...          # From console.anthropic.com
ELEVENLABS_API_KEY=...                # From elevenlabs.io settings
PLAYER_VOICE_ID=...                   # From ElevenLabs Voice Lab
FOOTBALL_API_KEY=...                  # From RapidAPI dashboard
SIMULATION_MODE=true                  # Set false for live match
   data
```